요구사항 분석

202344078 최예준 · 구현은 소스 확인 기준이며 운영 검증 완료를 뜻하지 않는다.

1. 시스템 및 사용자 역할

비회원은 탐색과 가입·로그인, 회원은 장바구니·가상 주문·본인 기록·주소·배달현황을 이용한다. 별도의 관리자나 판매자 역할은 구현되어 있지 않다.

ID	기능	상세 요구사항	상태
F01	회원가입	중복 아이디 조회 후 해시 비밀번호 저장	구현
F02	로그인·로그아웃	해시 검증, 세션 설정·정리; 기존 평문은 로그인 성공 시 해시 전환	구현
F03	검색·조회	음식점/메뉴 이름 LIKE 검색 및 음식점별 메뉴 조회	구현
F04	장바구니	세션에 메뉴 ID별 수량; 증가·감소·삭제 및 합계	구현
F05	가상 주문	주문과 항목 저장, commit 후 장바구니 초기화	구현
F06	내 주문 조회	목록·상세·배달현황에서 user_id 기준 데이터 제한	구현
F07	절약 집계	COUNT 및 SUM, COALESCE로 주문 없는 경우 0 처리	구현
F08	주소 관리	users.address 갱신; 주문 당시 delivery_address 복사	구현
F09	배달 상태	order_date 기준 20초 간격 5단계, 1초마다 화면 갱신	구현
F10	지도 표시	주문 음식점 JOIN, 중복·빈 주소 제외, 다중 마커와 정보창	구현
F11	거리 판정	주소 검색 완료 후 Haversine, 소수 둘째 자리·5km 가상 기준	구현
F12	주변 음식점 검색	사용자 주소 기반 자동 검색 API	계획
F13	후속 통계·경로	월별 목표, 도로 경로 및 기사 이동 표현	미구현

근거: app.py의 라우트 및 templates/ 화면. F12는 8주차 발표의 향후 계획, F13은 README의 확장 계획이며 현재 코드에서 구현을 확인하지 못했다.

비기능 요구사항과 예외 처리

안전성 요구와 현재 구현의 차이를 함께 기록한다.

영역	요구 및 현재 반영	보완할 부분
보안	해시 검증·파라미터 바인딩·주문 소유권 조건	고정 세션 비밀값, CSRF 방어 부재, debug 실행 설정 개선
무결성	PK/FK 정의, 주문 항목 단가·칼로리 및 주소 보관	트랜잭션 실패 예외·rollback 처리, 입력값과 중복 가입 경쟁 검증
사용성	빈 주문·빈 장바구니 안내, 주소 설정 유도	서버 측 공백 검증, 지도 실패 원인 안내, 화면 통일
유지보수	라우트 함수·템플릿 분리, 환경변수로 DB 설정	단일 app.py 분리, 의존성 버전 및 마이그레이션 이력 정리

2. 예외 처리 분석

상황	현재 동작	해석·주의
미로그인	보호 라우트에서 /login으로 이동	/order·/orders·/saving은 일부 DB 연결이 인증 확인보다 앞섬
빈 장바구니	/order에서 /cart로 이동	로그인 확인보다 빈 장바구니 검사가 먼저 수행됨
타 사용자 주문	id와 user_id 조건 불일치 시 /orders로 이동	주문 상세와 배달현황에 동일한 소유권 확인
사용자 주소 누락	사용자 없음 또는 빈 주소이면 /address로 이동	공백 문자열만 있는 주소는 주문 단계에서 차단하지 않음
음식점 주소 누락	NULL 또는 TRIM 결과가 빈 값이면 목록 제외	Geocoder 실패 음식점은 좌표 배열에 추가하지 않음
완료 화면 직접 접근	직전 주문 금액·칼로리 세션 값 없으면 /orders	항상 새 주문을 요구하는 일회성 토큰 구조는 아님

보안 관련 실제 비밀값은 보고서에 기재하지 않았다. "해시"는 비밀번호의 일방향 검증 방식이며 복호화 가능한 암호화와 구분한다.

데이터 사전 ①

자료형·제약은 저장된 SQL 기준. 운영 DB의 실제 DDL을 조회하지 않았다.

3. users / restaurants / menus

테이블·컬럼	타입·제약(확인 범위)	역할
users.id	INT / PK, AUTO_INCREMENT	회원 식별자
users.username	VARCHAR(50) / NOT NULL	로그인 ID; UNIQUE 선언은 초기 SQL에 없음
users.password	VARCHAR(255) / NOT NULL	신규 가입 해시; 과거 평문 호환 분기 존재
users.nickname	VARCHAR(50) / NOT NULL	표시 이름
users.created_at	DATETIME / DEFAULT CURRENT_TIMESTAMP	회원 생성 시각
users.address	VARCHAR(255) / NULL 허용	기본 배달 주소; DB/address.sql ALTER 확인
restaurants.id	INT / PK, AUTO_INCREMENT	음식점 식별자
restaurants.name	VARCHAR(100) / NOT NULL	음식점 이름
restaurants.category	VARCHAR(50) / NOT NULL	음식 분류
restaurants.description	VARCHAR(255) / NULL 허용	소개
restaurants.image	VARCHAR(255) / NULL 허용	이미지 경로용; 현재 화면에서 사용 미확인
restaurants.address	타입·제약 미확인 / 코드·SQL 사용	음식점 위치 검색용; 추가 DDL 없음
menus.id	INT / PK, AUTO_INCREMENT	메뉴 식별자
menus.restaurant_id	INT / NOT NULL, FK	restaurants.id 참조 선언
menus.name	VARCHAR(100) / NOT NULL	메뉴명
menus.price	INT / NOT NULL	현재 메뉴 단가(원)
menus.calories	INT / NOT NULL	현재 메뉴 칼로리(kcal)
menus.description	VARCHAR(255) / NULL 허용	메뉴 소개
menus.image	VARCHAR(255) / NULL 허용	이미지 경로용; 현재 화면에서 사용 미확인

근거: DB/schema.sql, DB/address.sql, DB/add menus.sql, app.py. address의 타입을 추정해 기입하지 않았다.

데이터 사전 ② 및 관계

주문 발생 당시의 값과 현재 메뉴 데이터를 구분한다.

테이블·컬럼	타입·제약(확인 범위)	역할
orders.id	INT / PK, AUTO_INCREMENT	주문 식별자
orders.user_id	INT / NOT NULL, FK	users.id 참조 선언
orders.order_date	DATETIME / DEFAULT CURRENT_TIMESTAMP	주문 시각·가상 배달 상태 기준
orders.total_price	INT / NOT NULL	주문 전체 금액
orders.total_calories	INT / NOT NULL	주문 전체 칼로리
orders.delivery_address	타입·제약 미확인 / 코드 사용	주문 시점 주소; 추가 DDL 없음
order_items.id	INT / PK, AUTO_INCREMENT	주문 항목 식별자
order_items.order_id	INT / NOT NULL, FK	orders.id 참조 선언
order_items.menu_id	INT / NOT NULL, FK	menus.id 참조 선언
order_items.quantity	INT / NOT NULL	주문 수량
order_items.price	INT / NOT NULL	주문 시점의 단위 가격
order_items.calories	INT / NOT NULL	주문 시점의 단위 칼로리

4. PK/FK 및 관계

5개 테이블의 id는 초기 스키마에 PK로 선언되어 있다. users → orders, restaurants → menus, orders → order_items, menus → order_items는 모두 1:N 관계이며 초기 SQL에 FK 선언이 있다. 운영 DB에 해당 제약이 그대로 적용되었는지는 미확인이다.

5. 주문과 음식점의 간접 연결

주문은 order_items → menus → restaurants를 거쳐 음식점에 연결된다. 주문 헤더에 restaurant_id를 추가로 보관하지 않는다. 한 주문에 여러 음식점의 메뉴가 포함될 수 있다. 라우트의 DISTINCT는 이름·주소 조합을 기준으로 중복을 제거한다.

확인 범위 및 해석
orders.delivery_address와 restaurants.address는 현재 코드에서 사용되지만 초기 schema.sql 및 별도 ALTER 파일에서 생성 정의를 찾지 못했다. users.address만 별도 ALTER 정의를 확인했다.

사용자 시나리오와 데이터 흐름

실제 실행 경로 및 데이터 보존 범위

6. 핵심 사용자 시나리오

순서	사용자 동작	시스템·데이터 흐름
1	가입·로그인	아이디 확인 및 비밀번호 검증 → user_id·nickname 세션 설정
2	탐색·메뉴 선택	restaurants / menus 조회 → 세션 cart에 menu_id별 수량
3	주소 설정	로그인 사용자의 users.address 조회·갱신
4	가상 주문	현재 메뉴 가격·칼로리 조회 → 합계 → 현재 사용자 주소 확인
5	주문 저장	orders 생성 → lastrowid로 order_items 연결 → commit
6	결과·기록 조회	장바구니 비우기 → 완료 화면 → 본인 내역·누적 절약 집계
7	배달현황 열기	주문 소유권 확인 → 음식점 목록 → 지도 좌표 검색·거리 표시

7. 저장 값과 계산 값

영구 저장: 사용자 주소, 주문 시각·합계·주소, 항목별 수량·단가·칼로리. 세션 저장: 로그인 정보, 장바구니, 직전 주문 합계. 브라우저 계산: 배달 상태, 지도 좌표, 음식점별 직선거리·가상 판정. 별도 cart·delivery·statistics 테이블은 없다.

8. 보존 범위와 분석 한계

주문 단가·칼로리·주소는 스냅샷으로 보관한다. 메뉴명과 음식점명은 상세 조회 시 현재 메뉴·음식점 테이블을 JOIN하므로 이름까지 완전한 이력 스냅샷인 것은 아니다. 여러 번 수행되는 메뉴 조회 사이의 동시 수정에 대한 잠금 처리도 확인되지 않는다.

DB/add menus.sql은 테스트 데이터 구성용이며 파괴적 초기화 문장이 포함되어 있다. 파일을 읽기만 했고 실행·제출하지 않았다. 실제 운영 DB의 행 수, 음식점 수, 성능 수치는 측정하지 않았다.

추가 확인 필요: 현재 DB의 SHOW CREATE TABLE 결과와 버전별 마이그레이션, 테스트 데이터의 주소·가격·칼로리 출처. 이 작업에서는 DB 연결이나 데이터 변경을 수행하지 않았다.